W4111_2025_002_1: Introduction to Databases: Homework 3A

Overview

Scope

The material in scope for this homework is:

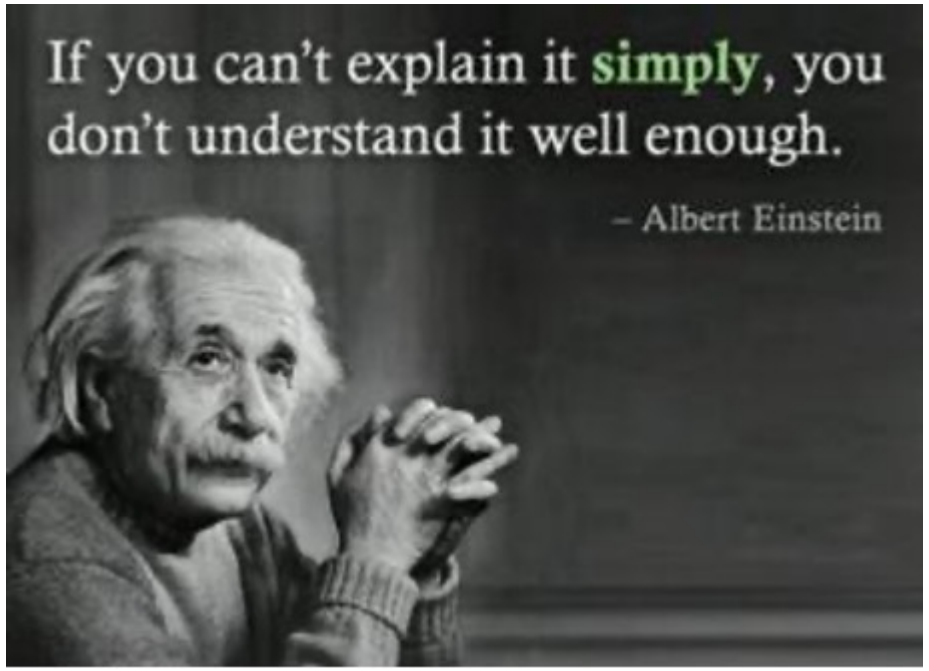
- The content of lectures:
  - This includes all of the meterial covered in the lectures.
  - All of the material in the slides from lectures 1, 2, and 3. Students should read slides not covered in the lecture.
  - Lecture 4 slides except for the slides after slide 88.
  - Lecture 5 slides from slide 1 to slide 57.
- The slides associated with the recommended textbook for
  - Chapter 1.
  - Chapter 2.
  - Chapter 3.
  - Chapter 4 slides 4.4 to 4.52 except for slide 4.35 (Transactions).

Submission Instructions

- Due date: 2025-Mar-09, 11:59 PM EDT on GradeScope.
- You submit on GradeScope. We will create a GradeScope submission for the homework.
- Your submission is a PDF of this notebook. You must tag the submission with locations in the PDF for each question. You must solve problems you experience producing a PDF including images. Please do not wait until the last minute.

There is a post/mega-thread on Ed Discussions that we will use to resolve questions and issues with respect to homework 3.

Brevity



Brevity

Students sometimes just write a lot of words hoping to get something right. We will deduct points if your answer is too long.

Initialization

Python Environment

In [33]:
```
import copy
```

In [34]:
```
import json
```

In [35]:
```
import pandas
```

In [36]:
```
# You should have installed the packages for previous homework assignments
#
```

```
import pymysql
import sqlalchemy
from sqlalchemy import create_engine
```

In [37]:
```
import numpy
```

In [38]:
```
# You have installed and configured ipython-sql for previous assignments.
# https://pypi.org/project/ipython-sql/
#
%load_ext sql
```

```
The sql extension is already loaded. To reload it, use:
  %reload_ext sql
```

In [39]:
```
# This is a hack to fix a version problem/incompatibility  with some of the packages and magics.
#
%config SqlMagic.style = '_DEPRECATED_DEFAULT'
```

In [40]:
```
# Make sure that you set these values to the correct values for your installation and
# configuration of MySQL
#
db_user = "root"
db_password = "dbuserdbuser"
```

In [41]:
```
# Create the URL for connecting to the database.
# Do not worry about the local_infile=1, I did that for wizard reasons that you should not have to use.
#
db_url = f"mysql+pymysql://{db_user}:{db_password}@localhost?local_infile=1"
```

In [42]:
```
# Initialize ipython-sql
#
%sql $db_url
```

In [43]:
```
# Your answer will be different based on the databases that you have created on your local MySQL instance.
#
%sql show tables from db_book
```

```
 * mysql+pymysql://root:***@localhost?local_infile=1
15 rows affected.
```

Out[43]:

Tables_in_db_book
advisor
classroom
course
course2
department
instructor
person
person_section
prereq
section
section2
student
takes
teaches
time_slot

In [44]:
```
default_engine = create_engine(db_url)
```

In [45]:
```
result_df = pandas.read_sql(
    "show tables from db_book", con=default_engine
)
result_df
```

Out[45]:

	Tables_in_db_book
0	advisor
1	classroom
2	course
3	course2
4	department
5	instructor
6	person
7	person_section
8	prereq
9	section
10	section2
11	student
12	takes
13	teaches
14	time_slot

Load Classic Models Dataset

You must load the `classicmodels` MySQL sample database.

You can use DataGrip and follow the process/tasks you did in HW 0 to load `db_book.` The page describing the dataset also links a tutorial on how to load the dataset. The installation file is a set of SQL DDL and DML statements. You ran a file with similar statements for HW 0.

The following query tests correct loading.

In [21]:

```
%%sql

use classicmodels;

with one as (select customerNumber,
```

```
                    (select customerName
                     from customers
                     where customers.customerNumber = orders.customerNumber) as customerName,
                    orderNumber
                from orders
                where (select country from customers where customers.customerNumber = orders.customerNumber) = 'Spain'),
two as (
    select
        *
    from one join orderdetails using(orderNumber)
)
select
    customerNumber, customerName,
    count(*) as noOfOrders,
    concat("$", format(sum(quantityOrdered*priceEach), 2)) as revenue
from two
group by customerNumber, customerName
having sum(quantityOrdered*priceEach) >= 50000
order by sum(quantityOrdered*priceEach) desc;
```

```
 * mysql+pymysql://root:***@localhost?local_infile=1
0 rows affected.
4 rows affected.
```

Out[21]:

customerNumber	customerName	noOfOrders	revenue
141	Euro+ Shopping Channel	259	$820,689.54
458	Corrida Auto Replicas, Ltd	32	$112,440.09
216	Enaco Distributors	23	$68,520.47
484	Iberia Gift Imports, Corp.	15	$50,987.85

Written Questions

IsA

Question

Briefly explain the concepts of:

- Inheritance/specialization/generalization.

- Incomplete/complete.
- Disjoint/overlapping.

Answer

Specialization is a top-down design process that divides an entity set into sub-groups that are have attributes or relationships that are distinct from other entities in the set. Generalization is a bottom-up design process that groups a number of entity sets with similar features/attributes into a larger, higher-level entity set.

Inheritance is when a lower level entity set shares (inherits) all the attributes and relationship participation of the higher level "parent" entity set.

If a specialization is complete, every instance of the parent class has one or more unique attributes that are not common to the parent class. In contrast, incomplete specialization is when not every instance of a parent class have unique attributes.

If a specialization is disjoint, an object could be a member of only one specialized subclass. In contrast, in an overlapping specialization, an object could be a member of more than one specialized subclass.

Views

Question

Views are a powerful concept in relational databases, and databases in general. Succinctly give 3 motivations for/reasons to use views.

Answer

1. A user does not need to see all the relations in a database. Some relations they do not care about/or should not see.
2. Views act as saved queries, which improves code readability and prevents the need to write the same query repeatedly.
3. Views allow changing table structures without affecting the underlying structures that other queries may rely on.

Types of Views

Question

Consider the following SQL statement that creates a view.

```
with one as (select customerNumber,
                    (select customerName
```

```
                    from customers
                    where customers.customerNumber = orders.customerNumber) as customerName,
                orderNumber
            from orders),
two as (
    select
        *
    from one join orderdetails using(orderNumber)
)
select
    customerNumber, customerName,
    count(*) as noOfOrders,
    concat("$", format(sum(quantityOrdered*priceEach), 2)) as revenue
from two
group by customerNumber, customerName
having sum(quantityOrdered*priceEach) >= 50000
order by sum(quantityOrdered*priceEach) desc;
```

Assume that:

1. The total number of orders over all customers is very large.
2. There are relatively few customers.
3. Many users frequently query (read) the view.
4. The rate of updating an order or creating and order is small.

What type of view would you use and why?

Answer

I would use a Materialized View here. Usually view definition represents saving an expression. Thus whenever the view is accessed the new relation needs to be recalculated.

Given our assumptions, we know that the total number of orders is very large, it would be expensive to query orders and orderdetails. This is further exacerbated by the fact that many users frequently query the view. Thus, if we use a materialized view that is calculated semi-frequently (e.g. once per day) we can avoid the expensive queries, and reduce latency for users reading the view. This is acceptable since the rate of updating orders is small.

View Expanson

Question

Consider the following SQL statements:

```
create or replace view hw3_customerSummary as select
    customerNumber, customerName, contactFirstName, contactLastName, phone as contactPhone, salesRepEmployeeNumber
from
    customers;

create or replace view hw3_customersAndSales as
select
    customerNumber, customerName, contactFirstName, contactLastName, contactPhone,
    employeeNumber as salesRepEmploueeNumber,
    concat(employees.lastName, ",", employees.firstName) as salesRepEmployeeName,
    email as saleRepEmployeeEmail
from
    hw3_customerSummary join employees on employeeNumber=salesRepEmployeeNumber;
```

Show the resulting query after view expansion for the following query.

```
select * from hw3_customersAndSales;
```

Answer

```
select *
from (select customerNumber,
             customerName,
             contactFirstName,
             contactLastName,
             contactPhone,
             employeeNumber                                        as salesRepEmploueeNumber,
             concat(employees.lastName, ',', employees.firstName) as salesRepEmployeeName,
             email                                                 as saleRepEmployeeEmail
      from (select customerNumber,
                   customerName,
                   contactFirstName,
                   contactLastName,
                   phone as contactPhone,
                   salesRepEmployeeNumber
            from customers) as hw3_customerSummary
               join employees on employeeNumber = salesRepEmployeeNumber) as hw3_customersAndSales;
```

View Updates

Question

Consider the following SQL DDL based on a modification of the `db_book` data model.

```
drop table if exists hw3_student;

create table if not exists db_book.hw3_student
(
    ID        varchar(5)  not null
        primary key,
    name      varchar(20) not null,
    dept_name varchar(20) null,
    tot_cred  decimal(3)  not null
);

create or replace view db_book.hw3_student_simple_1 as
    select ID, name, tot_cred from hw3_student;
create or replace view db_book.hw3_student_simple_2 as
    select ID, name from hw3_student;
```

For each views, state whether or not it is updateable and why.

Answer

db_book.hw3_student_simple_1 is updatable since all the not null columns are in the view. db_book.hw3_student_simple_2 is not updatable since it does not include tot_cred, which is defined as not null

Fun with Joins

Question

Using the default dataset UIBK - R, S, T, execute the following relational algebra expression (a full outer join):

$$S \bowtie T$$

The result is:

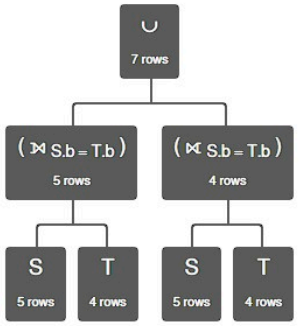
S.b	S.d	T.b	T.d
'a'	100	'a'	100
'b'	300	*null*	*null*
'c'	400	*null*	*null*
'd'	200	'd'	200
'e'	150	*null*	*null*
null	*null*	'f'	400
null	*null*	'g'	120

Write and execute a relational algebra expression that produces an equivalent result but does not use outer join.

Answer

Replace the statement and image below with your results.

```
(S ⟖ S.b=T.b T) ∪ (S ⟕ S.b=T.b T)
```



$$(S \bowtie_{S.b = T.b} T) \cup (S \ltimes_{S.b = T.b} T)$$

Execution time: 1 ms

S.b	S.d	T.b	T.d
'a'	100	'a'	100
'b'	300	*null*	*null*
'c'	400	*null*	*null*
'd'	200	'd'	200
'e'	150	*null*	*null*
null	*null*	'f'	400
null	*null*	'g'	120

Indexes

Question

Properly identifying and defining useful indexes is critical to database performance. Briefly explain:

1. Why do some databases automatically create indexes when the DDL defines primary, unique and foreign key constraints?
2. If indexes are useful, why doesn't the database engine automatically create every possible index?
3. The following is the DDL for `student` from the `db_book` database. For the purposes of this question, a "scenario" is a hypothesis about the size of tables and the types of queries users submit. Define a scenario for which creating an additional index on `student` would be beneficial and why.

```
create table if not exists db_book.student
(
    ID        varchar(5)  not null
        primary key,
    name      varchar(20) not null,
    dept_name varchar(20) null,
    tot_cred  decimal(3)  null,
    constraint student_ibfk_1
        foreign key (dept_name) references db_book.department (dept_name)
            on delete set null,
    check (`tot_cred` >= 0)
);

create index dept_name
    on db_book.student (dept_name);
```

Answer

1. Keys are mainly used to uniquely identify a specific entity. There is obvious benefit to indexing this since operations involving lookups will most likely be done with the key. Also, foreign keys and primary key pairs are clear candidates for indexes, and do not require complex calculations or optimizers to determine.
2. Indexes need to be stored, hence if every possible index is created automatically it may need significant amounts of disk space. Also, indexes need to be maintained when the DB is written to, decreasing write performance. Further, indexes are not useful in every context (e.g. analytics on every row of a relation).
3. Creating an additional index on ID would be beneficial since it is the Primary key for student, hence it would be used frequently in queries that involve student entities. One such scenario is when data on student demographics/statistics is needed, for example, to allocate budget for each department. This query would involve looking up each student from another relation (which the index would speed up) and accessing dept_name.

Codd's Rule 7

Question

Codd's **Rule 7: High-Level Insert, Update, and Delete Rule**

A database must support high-level insertion, updation, and deletion. This must not be limited to a single row, ...

Briefly explain and give examples of how SQL realizes/implements this rule.

Answer

SQL allows bulk INSERTs, UPDATEs, and DELETEs

For bulk insert, SQL allows inserting tables or inserting multiple rows simultaneously:

```
INSERT INTO students1
SELECT * FROM students2

INSERT INTO students1
VALUES ('12345', 'Bob', ...),
('12365', 'Andy', ...)
```

For bulk update, SQL allows performing an update over multiple rows of a relation, SQL also allows conditions

```
UPDATE students1
SET tot_cred = tot_cred*3
WHERE dept_name = 'Computer Science'
```

For bulk delete, SQL allows doing so and also can have conditions

```
DELETE FROM student1
WHERE tot_cred>30
```

Complex Attributes

Question

Consider the following Python code that processes some information about *Game of Thrones*.

In [15]:
```
fn = "s2025_episodes.json"

with open(fn, "r") as in_file:
    got_episodes = json.load(in_file)

for g in got_episodes:
    g["openingSequenceLocations"] = ";".join(g["openingSequenceLocations"])

got_episodes_df = pandas.DataFrame(got_episodes)
```

```
got_episodes_df
```

Out[15]:

	seasonNum	episodeNum	episodeTitle	episodeAirDate	openingSequenceLocations
0	1	1	Winter Is Coming	2011-04-17	King's Landing;Winterfell;The Wall;Pentos
1	1	2	The Kingsroad	2011-04-24	King's Landing;Winterfell;The Wall;Vaes Dothrak
2	1	3	Lord Snow	2011-05-01	King's Landing;Winterfell;The Wall;Vaes Dothrak
3	1	4	Cripples, Bastards, and Broken Things	2011-05-08	King's Landing;Winterfell;The Wall;Vaes Dothrak
4	1	5	The Wolf and the Lion	2011-05-15	King's Landing;The Eyrie;Winterfell;The Wall;V...
...	...	...	...	...	...
68	8	2	A Knight of the Seven Kingdoms	2019-04-21	Last Hearth;Winterfell;King's Landing
69	8	3	The Long Night	2019-04-28	Last Hearth;Winterfell;King's Landing
70	8	4	The Last of the Starks	2019-05-05	Last Hearth;Winterfell;King's Landing
71	8	5	The Bells	2019-05-12	Last Hearth;Winterfell;King's Landing
72	8	6	The Iron Throne	2019-05-19	Last Hearth;Winterfell;King's Landing

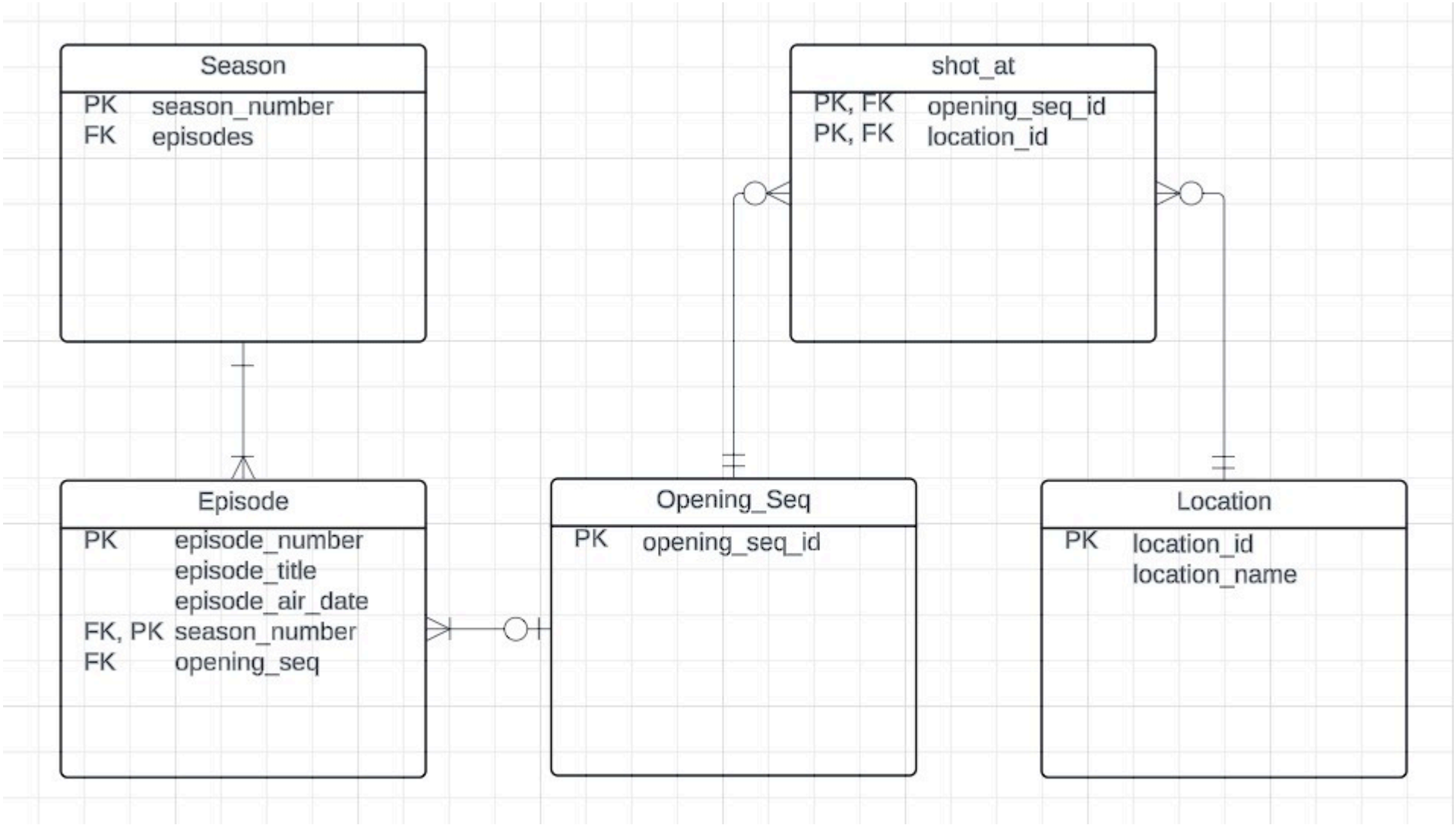
73 rows × 5 columns

Assume that I want to store the resulting dataframe in a relational database.

Draw a Crow's Foot logical ER diagram for the data model I should use. Replace the image below with your image. You may add explanatory notes if you wish.

Answer

Since season can be uniquely identified by its season number, I chose that as its primary key. A season may have one or many episodes. An Episode can be uniquely indentified by its episode number and season number, so I chose them as the Primay keys. An episode can have zero or one opening sequences, and an opening sequence may be in one or many episodes. I decided to model opening_seq as its own entity since a lot of episodes in a season share the same opening sequence, and opening sequence can contain additional data such as the locations it is shot at. This also allows the flexibility of having varying the opening sequence (e.g. for a throwback to an earlier season). I had to come up with a new unique opening_seq_id to identify it. An opening sequence can have 0 (CGI) or many filming location, and a filming location may have zero or many openings shot at it (a location maybe was used for episodes, but not opening). This is a many to many relationship so I decided to model the relationship as a shot_at entity.



Two Critical Concepts in Relationships

Question

Two critical concepts in entity relationships are *degree* and *cardinality.* Briefly explain the concepts.

Answer

The degree of a relationship set is how many entity sets are in one relationship. For example, the relationship "advisor" is of degree 2 since it relates instructors and students entity sets only. For a degree of 3, an example is the "proj_guide" relationship which involves instructor, student, and project.

The cardinality is the number of entities to which another entity can be associated via a relationship set. One to one, one to many, many to one, etc. are all types of cardinalitites. For example, in the relationship "advisor", the cardinality is one to many since a student may have only one advisor instructor, but an instructor may have many students they are advising.

Practical Questions

Metadata

Question

The following MySQL operation generates a basic description of a table.

In [16]:
```
%sql describe db_book.student;
```

```
 * mysql+pymysql://root:***@localhost?local_infile=1
4 rows affected.
```

Out[16]:

Field	Type	Null	Key	Default	Extra
ID	varchar(5)	NO	PRI	None	
name	varchar(20)	NO		None	
dept_name	varchar(20)	YES	MUL	None	
tot_cred	decimal(3,0)	YES		None	

Examine the `views` and `tables` in `information_schema.` Write a simple SQL DML query that produces the information above. Your answer does not have to have the same format.

Answer

Please place and execute your SQL statement below.

In [31]:
```
%%sql

use information_schema;

select COLUMN_NAME as Field, COLUMN_TYPE as Type, IS_NULLABLE as `Null`, COLUMN_KEY as `Key`, COLUMN_DEFAULT as `Default`, COLUMN_COMMENT a
limit 10;
```

```
 * mysql+pymysql://root:***@localhost?local_infile=1
0 rows affected.
4 rows affected.
```

Out[31]:

Field	Type	Null	Key	Default	Extra
dept_name	varchar(20)	YES	MUL	None	
ID	varchar(5)	NO	PRI	None	
name	varchar(20)	NO		None	
tot_cred	decimal(3,0)	YES		None	

Tricky SQL 1

Question

Consider the following query. **Note** that I used `limit 10` just to keep the answer short.

In [23]:

```
%%sql

use classicmodels;

with one as (
    select customerNumber, count(*) as numberOfOrders,
           sum(quantityOrdered*priceEach) as totalRevenue
    from orders join orderdetails using(orderNumber) group by customerNumber
)
select * from one
limit 10;
```

```
 * mysql+pymysql://root:***@localhost?local_infile=1
0 rows affected.
10 rows affected.
```

Out[23]:

customerNumber	numberOfOrders	totalRevenue
103	7	22314.36
112	29	80180.98
114	55	180585.07
119	53	158573.12
121	32	104224.79
124	180	591827.34
128	22	75937.76
129	21	66710.56
131	49	149085.15
141	259	820689.54

Modify the query so that the results contains:

- The customer's `customerName` and the customer's `country.`
- Only contains customers whose country is `Spain`

You must not use a JOIN.

Answer

In [24]:
```
%%sql

/* Place and execute your SQL here. */
with one as (
    select customerNumber, count(*) as numberOfOrders,
           sum(quantityOrdered*priceEach) as totalRevenue,
            (select customerName from customers where customers.customerNumber=orders.customerNumber) as customerName,
            (select country from customers where customers.customerNumber=orders.customerNumber) as country
    from orders join orderdetails using(orderNumber) group by customerNumber
)
select * from one where country='Spain'
limit 10;
```

```
 * mysql+pymysql://root:***@localhost?local_infile=1
5 rows affected.
```

Out[24]:

customerNumber	numberOfOrders	totalRevenue	customerName	country
141	259	820689.54	Euro+ Shopping Channel	Spain
216	23	68520.47	Enaco Distributors	Spain
344	13	46751.14	CAF Imports	Spain
458	32	112440.09	Corrida Auto Replicas, Ltd	Spain
484	15	50987.85	Iberia Gift Imports, Corp.	Spain

Tricky SQL 2

Question

Use the `db_book` database for this problem.

We want to compute a relation of the form $(course\_id, course\_title, prereq\_course\_id, prereq\_course\_title)$ that lists courses and their prereqs. If a course does not have a prereq, $prereq\_course\_id$ and $prereq\_course\_title$ should be `NULL.` If a course is not a prereq for any other course, it should appear in the result with $course\_id$ and $course\_title$ both `NULL.`

Write an SQL query that produces the table. To help, the result of running my solution is below.

23 rows

	course_id	course_title	prereq_id	prereq_course_title
1	BIO-101	Intro. to Biology	<null>	<null>
2	BIO-301	Genetics	BIO-101	Intro. to Biology
3	BIO-399	Computational Biology	BIO-101	Intro. to Biology
4	CS-101	Intro. to Computer Science	<null>	<null>
5	CS-190	Game Design	CS-101	Intro. to Computer Science
6	CS-315	Robotics	CS-101	Intro. to Computer Science
7	CS-319	Image Processing	CS-101	Intro. to Computer Science
8	CS-347	Database System Concepts	CS-101	Intro. to Computer Science
9	EE-181	Intro. to Digital Systems	PHY-101	Physical Principles
10	FIN-201	Investment Banking	<null>	<null>
11	HIS-351	World History	<null>	<null>
12	MU-199	Music Video Production	<null>	<null>
13	PHY-101	Physical Principles	<null>	<null>
14	<null>	<null>	BIO-301	Genetics
15	<null>	<null>	BIO-399	Computational Biology
16	<null>	<null>	CS-190	Game Design
17	<null>	<null>	CS-315	Robotics
18	<null>	<null>	CS-319	Image Processing
19	<null>	<null>	CS-347	Database System Concepts
20	<null>	<null>	EE-181	Intro. to Digital Systems
21	<null>	<null>	FIN-201	Investment Banking
22	<null>	<null>	HIS-351	World History
23	<null>	<null>	MU-199	Music Video Production

Course-Prereqs

Answer

In [20]:
```
%%sql

/* Place and execute your SQL here. */
use db_book;

select course_and_prereq.course_id, course_and_prereq.title as course_title, prereq_id, course.title as prereq_course_title from
(select course.course_id, title, prereq_id from course left join prereq on course.course_id = prereq.course_id) as course_and_prereq
```

```
left join course on course_and_prereq.prereq_id = course.course_id

union

select prereqs_and_courses_right.course_id as course_id, title as course_title, prereq_id, prereq_course_title from
(select prereq.course_id as course_id, course.course_id as prereq_id, title as prereq_course_title from prereq right join course on course.
left join course on course.course_id = prereqs_and_courses_right.course_id
```

```
 * mysql+pymysql://root:***@localhost?local_infile=1
0 rows affected.
23 rows affected.
```

Out[20]:

course_id	course_title	prereq_id	prereq_course_title
BIO-101	Intro. to Biology	None	None
BIO-301	Genetics	BIO-101	Intro. to Biology
BIO-399	Computational Biology	BIO-101	Intro. to Biology
CS-101	Intro. to Computer Science	None	None
CS-190	Game Design	CS-101	Intro. to Computer Science
CS-315	Robotics	CS-101	Intro. to Computer Science
CS-319	Image Processing	CS-101	Intro. to Computer Science
CS-347	Database System Concepts	CS-101	Intro. to Computer Science
EE-181	Intro. to Digital Systems	PHY-101	Physical Principles
FIN-201	Investment Banking	None	None
HIS-351	World History	None	None
MU-199	Music Video Production	None	None
PHY-101	Physical Principles	None	None
None	None	BIO-301	Genetics
None	None	BIO-399	Computational Biology
None	None	CS-190	Game Design
None	None	CS-315	Robotics
None	None	CS-319	Image Processing
None	None	CS-347	Database System Concepts
None	None	EE-181	Intro. to Digital Systems
None	None	FIN-201	Investment Banking
None	None	HIS-351	World History
None	None	MU-199	Music Video Production

ER Diagram and DDL

Question

Consider the following scenario. There are 6 entity types.

1. `BaseProduct` that has attributes:
   - `product_id`
   - `product_name`
   - `product_description`
   - `price`
2. `DurableProduct` **isA** `BaseProduct` and has the attributes:
   - `height`
   - `width`
   - `depth`
   - `weight`
3. `EntertainmentProduct` **isA** a `BaseProduct` and has the attributes:
   - `copyright_date`
   - `version_no`
   - `artist_first_name`
   - `artist_last_name`
4. `MusicProduct` **isA** an `EntertainmentProduct` an has the attributes:
   - `no_of_seconds`
   - `genre`
5. `BookProduct` **isA** an `EntertainmentProduct` and has the attributes:
   - `no_of_pages`
   - `printing_no`
6. `Supplier` has the attributes:
   - `supplier_id`
   - `supplier_name`

A product is either a `DurableProduct,` an `EntertainmentProduct` or just a `BaseProduct.` There are no products that are both durable and entertainment. All `EntertainmentProducts` are either a book or a music product but not both. Every product has *exactly* one `Supplier.` A `Supplier` may supply several products.

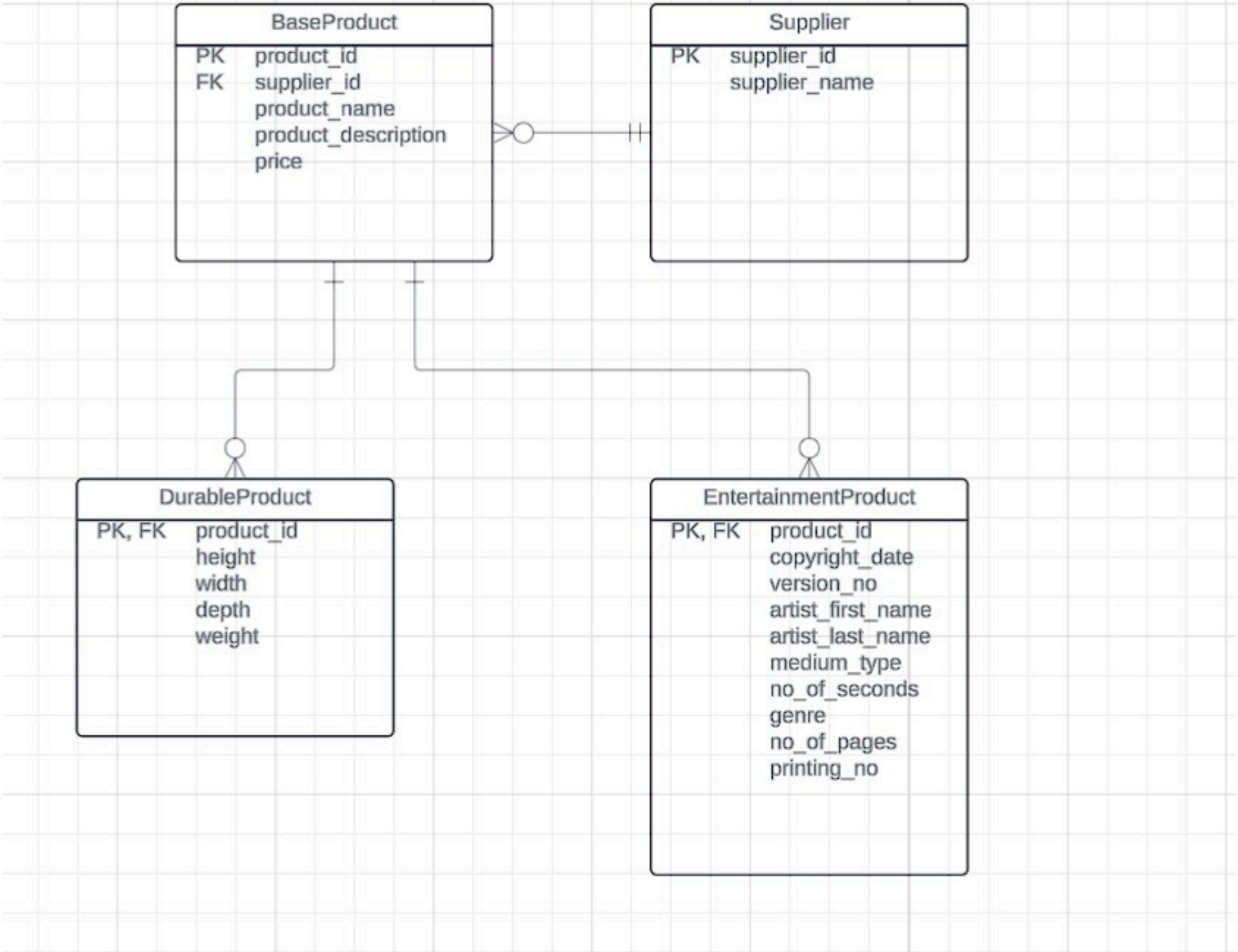
Draw a Crow's Foot ER Diagram for the logical model for this scenario. You may have to add attributes to the lists above to implement define the model. Specify whether you are choosing the one, two or three table design. You should also specify which views you would create.

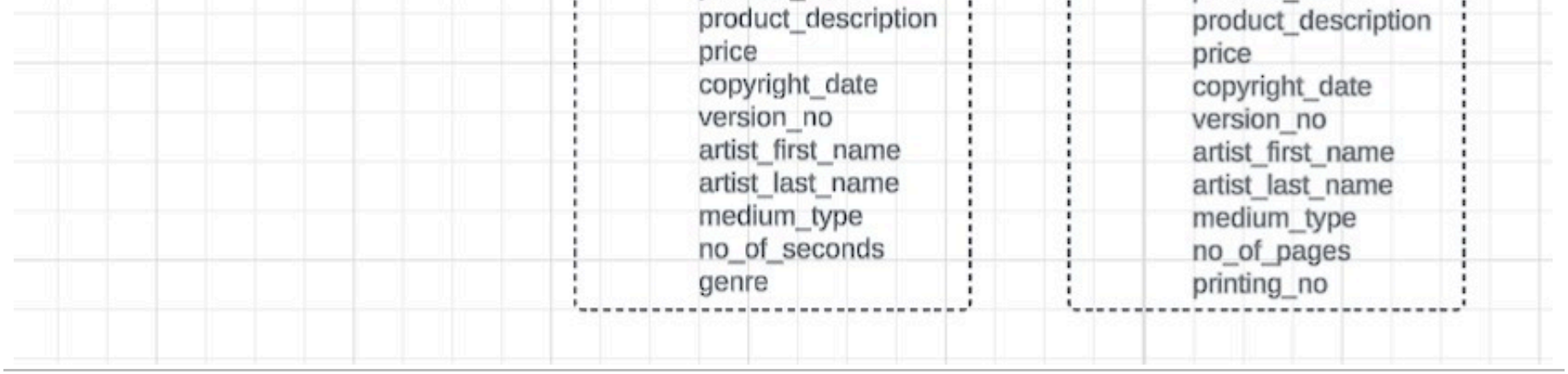
You should indicate **all foreign keys.**

Answer

Place a screenshot of you diagram here and replace this image, which is some symbols.

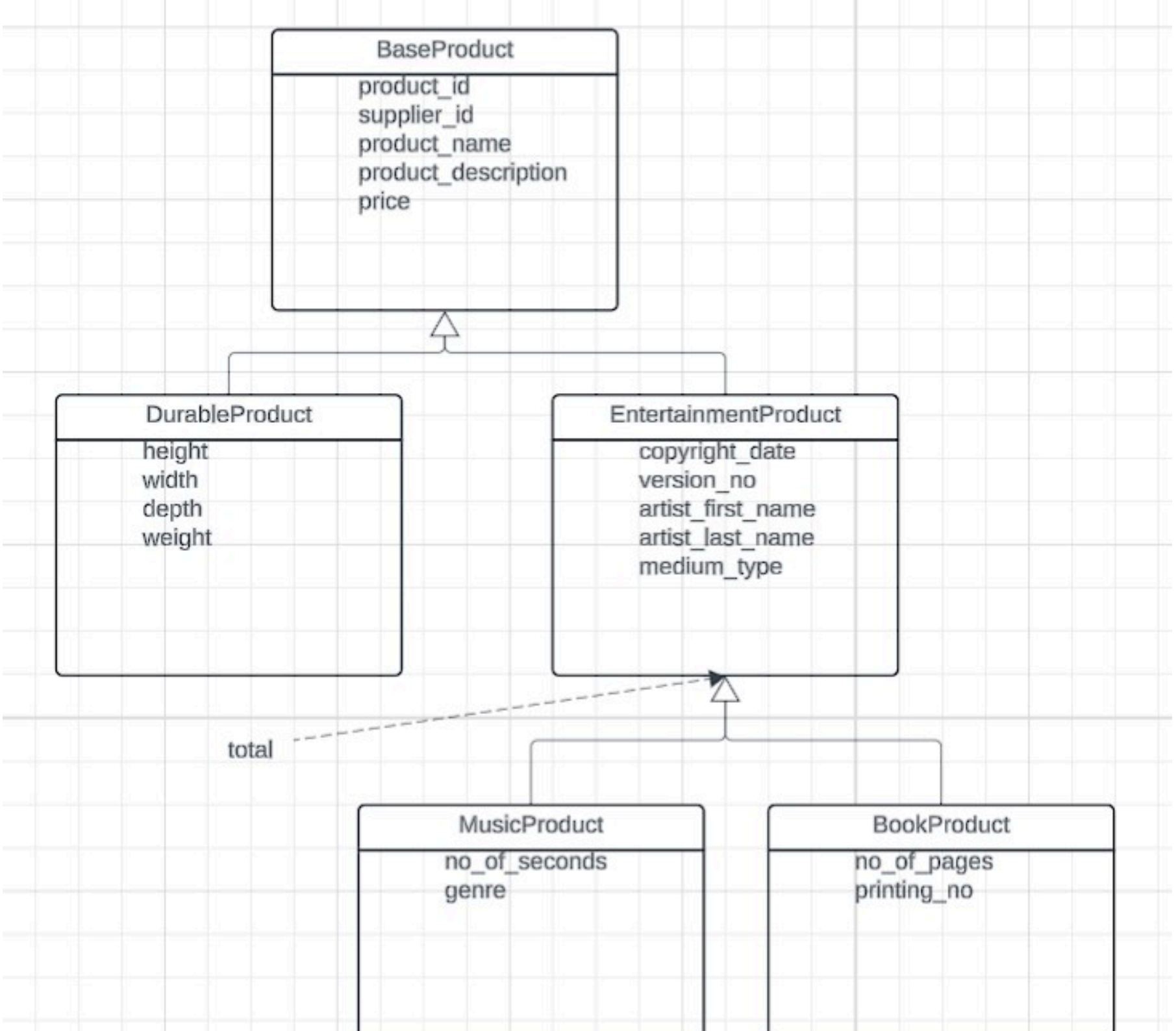
For the BaseProduct, DurableProduct, EntertainmentProduct specialization I used a three table design. For the EntertainmentProduct, MusicProduct, BookProduct specialization I used a one table design, with MusicProduct and BookProduct as views. To allow distinguishing BookProduct and MusicProduct, I created a new attribute 'medium_type' in EntertainmentProduct.


BaseProduct
PK product_id
FK supplier_id
product_name
product_description
price
Supplier
PK supplier_id
supplier_name
DurableProduct
PK, FK product_id
height
width
depth
weight
EntertainmentProduct
PK, FK product_id
copyright_date
version_no
artist_first_name
artist_last_name
medium_type
no_of_seconds
genre
no_of_pages
printing_no
MusicProduct
PK, FK product_id
product_name
BookProduct
PK, FK product_id
product_name



ER diagram

There is no overlap between DurableProduct and EntertainmentProduct, and no overlap between MusicProduct and BookProduct.


BaseProduct
product_id
supplier_id
product_name
product_description
price
DurableProduct
height
width
depth
weight
EntertainmentProduct
copyright_date
version_no
artist_first_name
artist_last_name
medium_type
total
MusicProduct
no_of_seconds
genre
BookProduct
no_of_pages
printing_no

Inheritance

Please place and execute your SQL statement below.

In [49]:

```
%%sql

use db_book;

drop table if exists Supplier;
drop table if exists BaseProduct;
drop table if exists DurableProduct;
drop table if exists EntertainmentProduct;

create table Supplier(
    supplier_id CHAR(10) PRIMARY KEY,
    supplier_name VARCHAR(100) NOT NULL
);

create table BaseProduct (
    product_id CHAR(10) PRIMARY KEY,
    supplier_id CHAR(10) NOT NULL REFERENCES Supplier(supplier_id),
    product_name VARCHAR(100) NOT NULL,
    product_description TEXT,
    price DECIMAL NOT NULL CHECK (price > 0)
);

create table DurableProduct (
    product_id CHAR(10) PRIMARY KEY REFERENCES BaseProduct(product_id),
    height FLOAT NOT NULL CHECK (height > 0),
    width FLOAT NOT NULL CHECK (width > 0),
    depth FLOAT NOT NULL CHECK (depth > 0),
    weight FLOAT NOT NULL CHECK (weight > 0)
);

create table EntertainmentProduct (
    product_id CHAR(10) PRIMARY KEY REFERENCES BaseProduct(product_id),
    copyright_date DATE,
    version_no NUMERIC CHECK (version_no > 0),
    artist_first_name VARCHAR(100),
    artist_last_name VARCHAR(100),
    medium_type ENUM('book', 'music') NOT NULL,
```

```
    no_of_seconds INT CHECK (no_of_seconds > 0),
    genre VARCHAR(100),
    no_of_pages INT CHECK (no_of_pages > 0),
    printing_no INT CHECK (printing_no > 0)
);

create or replace view MusicProduct as select
    BaseProduct.product_id,
    BaseProduct.supplier_id,
    BaseProduct.product_name,
    BaseProduct.product_description,
    BaseProduct.price,
    EntertainmentProduct.copyright_date,
    EntertainmentProduct.version_no,
    EntertainmentProduct.artist_first_name,
    EntertainmentProduct.artist_last_name,
    EntertainmentProduct.medium_type,
    EntertainmentProduct.no_of_seconds,
    EntertainmentProduct.genre
from
    EntertainmentProduct, BaseProduct
where EntertainmentProduct.product_id = BaseProduct.product_id AND medium_type = 'book';

create or replace view MusicProduct as select
    BaseProduct.product_id,
    BaseProduct.supplier_id,
    BaseProduct.product_name,
    BaseProduct.product_description,
    BaseProduct.price,
    EntertainmentProduct.copyright_date,
    EntertainmentProduct.version_no,
    EntertainmentProduct.artist_first_name,
    EntertainmentProduct.artist_last_name,
    EntertainmentProduct.medium_type,
    EntertainmentProduct.no_of_pages,
    EntertainmentProduct.printing_no
from
    EntertainmentProduct, BaseProduct
where EntertainmentProduct.product_id = BaseProduct.product_id AND medium_type = 'music';
```

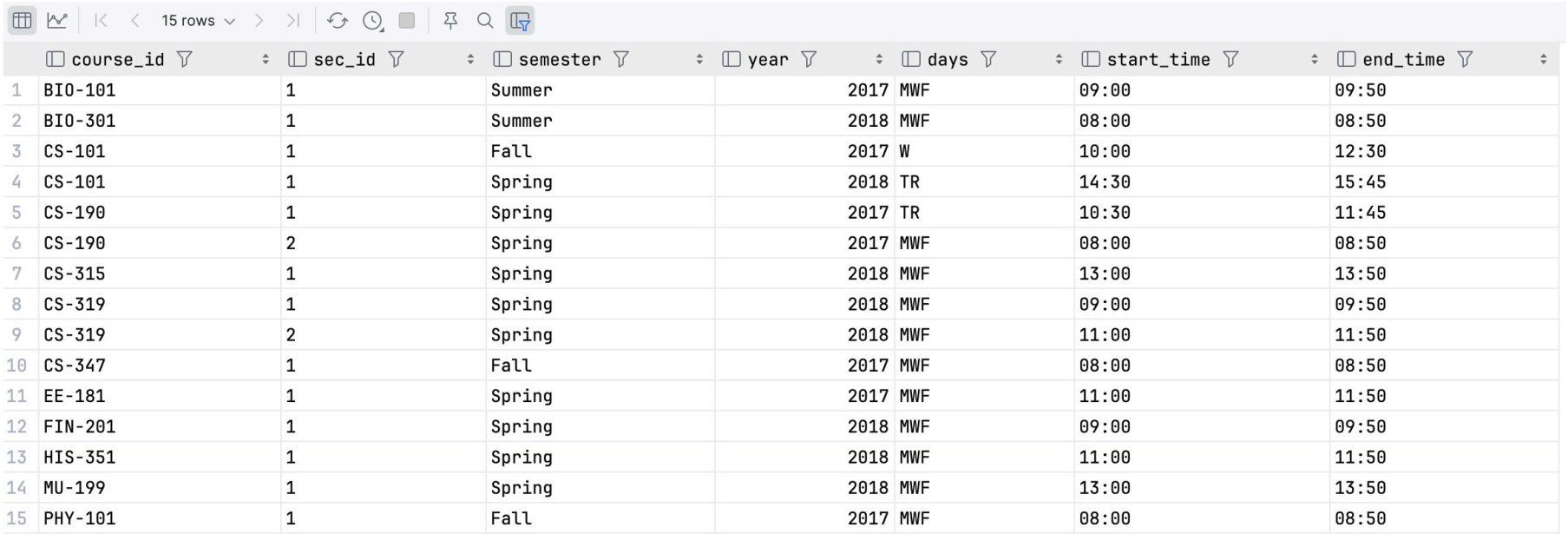
```
 * mysql+pymysql://root:***@localhost?local_infile=1
0 rows affected.
0 rows affected.
0 rows affected.
0 rows affected.
0 rows affected.
0 rows affected.
0 rows affected.
0 rows affected.
0 rows affected.
0 rows affected.
0 rows affected.
```

Out[49]: []

I Told You that You Would Have to Look Up Functions SQL Question

Question

The following result table uses `section` and `time_slot` from `db_book.`



15 rows

	course_id	sec_id	semester	year	days	start_time	end_time
1	BIO-101	1	Summer	2017	MWF	09:00	09:50
2	BIO-301	1	Summer	2018	MWF	08:00	08:50
3	CS-101	1	Fall	2017	W	10:00	12:30
4	CS-101	1	Spring	2018	TR	14:30	15:45
5	CS-190	1	Spring	2017	TR	10:30	11:45
6	CS-190	2	Spring	2017	MWF	08:00	08:50
7	CS-315	1	Spring	2018	MWF	13:00	13:50
8	CS-319	1	Spring	2018	MWF	09:00	09:50
9	CS-319	2	Spring	2018	MWF	11:00	11:50
10	CS-347	1	Fall	2017	MWF	08:00	08:50
11	EE-181	1	Spring	2017	MWF	11:00	11:50
12	FIN-201	1	Spring	2018	MWF	09:00	09:50
13	HIS-351	1	Spring	2018	MWF	11:00	11:50
14	MU-199	1	Spring	2018	MWF	13:00	13:50
15	PHY-101	1	Fall	2017	MWF	08:00	08:50

Using Functions

Write an SQL query that produces the table. You will have to look for some SQL functions to use. You can use the web, ChatGPT, etc. to find the documentation on the functions.

The order of the multiple day classes must be "MWF" or "TR." Other orders, e.g. "FMW" are not acceptable.

In [ ]:

Answer

In [22]:
```
%%sql
/*
    Write and execute your answer.
*/
use db_book;

select course_id, sec_id, semester, year, days, start_time, end_time
from section
         join
     (select distinct time_slot_id, days, start_time, end_time
      from (
               (select time_slot_id,
                       day,
                       TIME_FORMAT(CONCAT(start_hr, ':', start_min), '%H:%i') as start_time,
                       TIME_FORMAT(CONCAT(end_hr, ':', end_min), '%H:%i')     as end_time
                from time_slot) as time_slots_with_time
                  natural join
                  (select time_slot_id,
                          GROUP_CONCAT(day ORDER BY
                                       CASE day
                                           WHEN 'M' THEN 1
                                           WHEN 'T' THEN 2
                                           WHEN 'W' THEN 3
                                           WHEN 'R' THEN 4
                                           WHEN 'F' THEN 5
                                           ELSE 6
                                           END
                                       SEPARATOR '') as days
                   from time_slot
                   GROUP BY time_slot_id) as time_slot_with_days
               )) as formatted_time_slot
     on section.time_slot_id = formatted_time_slot.time_slot_id
```

```
 * mysql+pymysql://root:***@localhost?local_infile=1
0 rows affected.
15 rows affected.
```

Out[22]:

course_id	sec_id	semester	year	days	start_time	end_time
BIO-101	1	Summer	2017	MWF	09:00	09:50
BIO-301	1	Summer	2018	MWF	08:00	08:50
CS-101	1	Fall	2017	W	10:00	12:30
CS-101	1	Spring	2018	TR	14:30	15:45
CS-190	1	Spring	2017	TR	10:30	11:45
CS-190	2	Spring	2017	MWF	08:00	08:50
CS-315	1	Spring	2018	MWF	13:00	13:50
CS-319	1	Spring	2018	MWF	09:00	09:50
CS-319	2	Spring	2018	MWF	11:00	11:50
CS-347	1	Fall	2017	MWF	08:00	08:50
EE-181	1	Spring	2017	MWF	11:00	11:50
FIN-201	1	Spring	2018	MWF	09:00	09:50
HIS-351	1	Spring	2018	MWF	11:00	11:50
MU-199	1	Spring	2018	MWF	13:00	13:50
PHY-101	1	Fall	2017	MWF	08:00	08:50

In [ ]: